INTERNSHIP PROJECT REPORT

SIM-003 – MQTT Telemetry Simulator

Detailed Internship Project Documentation

Vishal Kumar
B.Tech Electronics & Communication Engineering

Table of Contents

1. 1. Executive Summary
2. 2. Introduction
3. 3. Project Background and Business Need
4. 4. Problem Statement
5. 5. Project Objectives
6. 6. Scope
7. 7. Functional Requirements
8. 8. Non-Functional Requirements
9. 9. Technology Stack
10. 10. High-Level Architecture
11. 11. Module-Level Design
12. 12. Telemetry Data Model
13. 13. Normal Scenario
14. 14. Delayed Telemetry Scenario
15. 15. Duplicate Packet Scenario
16. 16. Out-of-Range Scenario
17. 17. Missing Timestamp Scenario
18. 18. Spoofed Identity Scenario
19. 19. Replay Attack Scenario
20. 20. Service Layer and Validation
21. 21. FastAPI API Design
22. 22. API Example
23. 23. Streamlit Dashboard
24. 24. Command-Line Generation
25. 25. Testing Strategy
26. 26. QA Results
27. 27. Security Design
28. 28. Documentation and Evidence
29. 29. Demo and Verification
30. 30. GitHub and Version Control
31. 31. Project Limitations
32. 32. Future Improvements

33. 33. Internship Learning Outcomes

34. 34. Final Project Status

35. 35. Conclusion

1. Executive Summary

SIM-003 – MQTT Telemetry Simulator is a Python-based local simulation project developed to generate battery telemetry events similar to telemetry that could be produced by a Battery Management System (BMS). The project is intended to support telemetry trust, anomaly testing and replay-attack concept validation without requiring real BMS hardware, a physical sensor setup, a real MQTT broker or live network traffic.

The simulator provides seven defined scenarios: normal telemetry, delayed telemetry, duplicate packets, out-of-range values, missing timestamp, spoofed source identity and replay attack. The generated events contain battery measurements together with simulation and event metadata. A service layer is used by both the FastAPI API and Streamlit interface so that the same scenario-generation logic can be exercised through different user interfaces.

The final implementation includes telemetry models, a battery-data generator, scenario processing, a service layer, FastAPI endpoints, a Streamlit dashboard, automated tests and project documentation. The recorded final automated test result was 19 passed and 0 failed, with 71% coverage. The project also includes QA, security, API, architecture, design, verification and release documentation.

2. Introduction

Battery telemetry is important for monitoring battery systems because parameters such as voltage, current, temperature, state of charge and state of health can be used to understand battery operating conditions. For software development and cybersecurity testing, it is useful to have a repeatable way to produce telemetry without depending on physical hardware.

This project addresses that need through a local telemetry simulator. Instead of connecting to a real battery pack, the application generates structured telemetry records in software. The same event structure can then be passed through normal and abnormal scenarios so that a tester can observe how different conditions are represented.

The project was also designed around telemetry trust concepts. In particular, replay attacks and spoofed identities are represented as controlled simulation cases. This makes it possible to demonstrate anomaly flags and metadata without generating real malicious network traffic.

3. Project Background and Business Need

The assignment describes the simulator as a supporting component for telemetry trust and cybersecurity testing. The intended users include developers and testers working with telemetry trust architecture and replay-attack detection concepts.

A real BMS setup can require hardware, sensors, communication interfaces and an operational battery system. Such a setup is not always available during early software development. A local simulator reduces this dependency by providing predictable software-generated events.

The simulator is therefore useful for development activities such as API testing, scenario testing, dashboard development, data-format validation, anomaly handling and demonstration. It does not replace a real BMS or a production MQTT infrastructure; its purpose is controlled simulation and testing.

4. Problem Statement

The main problem is the need for a controlled source of battery telemetry containing both normal and abnormal events. A testing system needs to observe how telemetry behaves when messages are delayed, duplicated, malformed or associated with suspicious identities.

Without a simulator, these conditions may require manual data preparation or access to hardware. The project provides a repeatable software mechanism for producing these cases and exposing them through an API and user interface.

The assignment explicitly limits the simulator to local software simulation. It does not require implementation of a real MQTT broker, real BMS hardware, physical sensors or real network attack traffic.

5. Project Objectives

The project objectives are:

1. Generate simulated battery telemetry.

2. Represent telemetry as structured JSON-compatible data.

3. Support all seven required scenarios.

4. Mark abnormal scenarios with anomaly indicators.

5. Simulate delayed, duplicate, out-of-range, missing-timestamp, spoofed-identity and replay conditions.

6. Provide FastAPI endpoints for programmatic generation.

7. Provide a Streamlit interface for interactive scenario selection.

8. Provide CSV export from the dashboard.

9. Validate the implementation through automated tests.

10. Maintain supporting technical, QA and security documentation.

6. Scope

In scope:

• Battery telemetry generation.

• Scenario-based event generation.

• Event metadata such as event_id, sequence_number, topic, QoS and simulated flag.

• FastAPI endpoints.

• Streamlit dashboard.

• Automated tests with pytest.

• Coverage measurement.

• QA and security documentation.

• Demonstration and evidence preparation.

Out of scope:

• Real MQTT broker deployment.

• Physical BMS integration.

• Physical sensor acquisition.

• Real network traffic.

• Production-grade cybersecurity controls.

• A real battery pack or charging system.

• Proprietary BMS communication protocols.

7. Functional Requirements

The assignment defines seven functional telemetry scenarios. The simulator must generate normal events and controlled anomaly cases. Delayed events use a configurable delay from 1 to 300 seconds. Duplicate events use the same sequence number. Out-of-range values include voltage above 60 V, temperature above 60 °C and current above 500 A. Missing-timestamp events omit the timestamp. Spoofed identity events use a simulated identity. Replay events use an older timestamp while recording a current send time.

The output is structured around an MQTT-style topic and JSON payload, with QoS metadata. The project also exposes the generation function through FastAPI and presents it through Streamlit.

8. Non-Functional Requirements

The assignment specifies a local/no-broker design, command-line support, a Streamlit user interface, a FastAPI interface, Pydantic v2 validation and pytest-based testing.

The implementation therefore focuses on modular Python code and testability. The generator and scenario logic are kept separate from the API and dashboard so that the same core logic can be reused.

9. Technology Stack

Python 3.10+ is the project language. Pydantic v2 is used for structured telemetry validation. FastAPI provides the REST API and Uvicorn runs the API server. Streamlit provides the interactive dashboard and Pandas is used to display generated event data and create CSV output. pytest is used for automated testing and coverage is measured with pytest-cov. Git and GitHub are used for source control and project evidence.

10. High-Level Architecture

The high-level data flow is:

User / Tester

↓

FastAPI API or Streamlit UI

↓

Service Layer

↓

Scenario Generator

↓

Battery Telemetry Generator

↓

Structured Event List

↓

JSON response / Streamlit table / CSV

The core scenario logic is centralized so that API and dashboard users receive consistent event structures. The simulator uses the topic pattern battery/telemetry/{battery_id}. Events include simulated=true to distinguish them from real telemetry.

11. Module-Level Design

models.py – Defines the Pydantic BatteryData model containing battery_id, voltage, current, temperature, SOC, SOH and timestamp.

generator.py – Generates battery telemetry values and returns validated BatteryData records.

scenarios.py – Contains the seven scenario names and the generate_scenario_events function. It adds event metadata and applies scenario-specific transformations.

service.py – Provides a validation-oriented service function generate_events used by the API and Streamlit application.

api.py – Defines the FastAPI application and REST endpoints.

app.py – Defines the Streamlit dashboard.

main.py – Provides command-line generation of telemetry and saving of sample data.

test/ – Contains automated tests for generator, API and service behaviour.

12. Telemetry Data Model

The core BatteryData structure contains:

• battery_id – identifies the simulated battery.

• voltage – simulated battery voltage.

• current – simulated current.

• temperature – simulated battery temperature.

• soc – state of charge.

• soh – state of health.

• timestamp – telemetry event time.

Scenario processing adds:

• event_id – event identifier such as EVT-001.

• sequence_number – event sequence.

• topic – MQTT-style telemetry topic.

• qos – simulated QoS value.

• simulated – true for generated simulation data.

• anomaly – scenario/anomaly indicator.

• sent_at – used where a separate send time is needed, such as delayed or replay cases.

13. Normal Scenario

The normal scenario produces standard telemetry without an anomaly flag. Events retain the generated battery measurements and standard metadata. It is the baseline case used to verify that the simulator can generate valid telemetry.

The normal scenario is also used by the API sample endpoint and the Streamlit dashboard demonstration.

14. Delayed Telemetry Scenario

The delayed scenario simulates telemetry that is generated at one time but sent later. The event timestamp is retained as the original telemetry time and sent_at is calculated by adding the configured delay.

The project accepts delay_seconds and validates that the delay is not negative. The assignment defines the intended delay range as 1–300 seconds for the delayed scenario. The anomaly indicator is DELAYED.

15. Duplicate Packet Scenario

The duplicate scenario represents a repeated telemetry packet. The implementation keeps the event structure but sets sequence_number to 1 for the generated duplicate records. The anomaly indicator is DUPLICATE.

The purpose is to provide a controlled input for duplicate-detection logic without sending duplicate packets over a real network.

16. Out-of-Range Scenario

The out-of-range scenario intentionally generates values beyond the defined limits. The implementation uses voltage 65.0 V, temperature 75.0 °C and current 600.0 A. The anomaly indicator is OUT_OF_RANGE.

These values are simulation values for testing and should not be interpreted as measurements from a real battery system.

17. Missing Timestamp Scenario

The missing-timestamp scenario removes the timestamp field from the generated event and marks the event with MISSING_TIMESTAMP. It allows validation of consumers that expect a telemetry timestamp and should identify incomplete records.

18. Spoofed Identity Scenario

The spoofed identity scenario changes the battery identifier to a simulated identity using the SIM-SPOOF prefix. The event topic is updated to match the simulated battery identity and the anomaly is marked SPOOFED_ID.

This is a software representation of an identity anomaly and does not perform a real identity attack.

19. Replay Attack Scenario

The replay scenario is one of the central cybersecurity demonstrations. The generated telemetry timestamp is moved backward by 300 seconds, while sent_at is set to the current send time. The event is marked REPLAY_DETECTED.

The combination of an old telemetry timestamp and a current send timestamp makes the simulated replay condition visible in the generated JSON. This scenario was tested through the FastAPI Swagger interface and Streamlit dashboard during the demo preparation.

20. Service Layer and Validation

The service layer provides a common generate_events function. It validates that battery_id is provided, count is greater than zero and delay_seconds is not negative. It then delegates scenario creation to generate_scenario_events.

Centralizing these checks avoids duplicating validation logic in the API and dashboard. Invalid scenario names are also checked against the supported scenario list by the API.

21. FastAPI API Design

The FastAPI application is titled MQTT Telemetry Simulator. The main endpoints are:

GET / – confirms that the API is running.

GET /health – returns a healthy status.

GET /scenarios – returns the seven supported scenarios.

POST /generate-events – accepts generation parameters and returns telemetry events.

GET /sample – returns a sample normal telemetry event.

The /generate-events endpoint accepts battery_id, scenario, count and delay_seconds. It returns the requested scenario, number of generated events and the event list.

22. API Example

Example request:

POST /generate-events

{

"battery_id": "BAT001",

"scenario": "replay_attack",

"count": 2,

"delay_seconds": 10

}

A replay response contains the simulated event data, an older timestamp, a current sent_at value, anomaly=REPLAY_DETECTED and simulated=true.

23. Streamlit Dashboard

The Streamlit application provides a simple interactive interface. The sidebar contains Battery ID, Select Scenario, Number of Events and Delay controls. Clicking Generate Events calls the service layer.

The generated records are converted into a Pandas DataFrame and displayed as a telemetry event stream. The dashboard also provides CSV download and summary metrics for total events, anomalies and scenario name.

The dashboard lists all seven available scenarios so that a tester can quickly select a test condition.

24. Command-Line Generation

The project includes a main.py command-line path for generating telemetry and saving JSON sample data. The application accepts a count argument, generates records and writes sample_data/battery_data.json.

The assignment also documents a package-style CLI example. The implemented project uses main.py-based argument parsing rather than claiming that the exact package command is implemented.

25. Testing Strategy

Testing covers the generator, API and service layers. The test suite verifies normal generation, scenario behaviour, API health, scenario listing, sample response, event generation, replay behaviour and validation errors.

The project uses pytest and pytest-cov. The final recorded result was 19 passed, 0 failed and 71% total coverage.

26. QA Results

The Week 8 QA work recorded 19 automated tests passing with no failed tests. A single deprecation warning related to the Starlette TestClient and httpx was observed. It did not affect the test result.

The QA report also records security-oriented checks for simulated flags, replay indicators, spoofed simulated identities and absence of real MQTT network traffic.

27. Security Design

The simulator is designed as a safe local simulation. Generated records contain simulated=true. Scenario names explicitly represent abnormal conditions rather than performing real attacks.

Replay, duplicate, spoofed identity and out-of-range cases are represented at the data level. This allows application logic to be tested without interacting with real devices or networks.

28. Documentation and Evidence

The project documentation set includes architecture documentation, design documentation, API documentation, QA report, security report, demo evidence, demo script, final verification, release notes, research notes, patent notes, presentation material and project closure material.

The assignment evidence checklist also identifies GitHub repository, branch, pull request, README, API document, verification report, security report, test report, coverage report, screenshots, demo video, presentation, research notes, patent notes and release notes as relevant evidence.

29. Demo and Verification

The recorded demo flow includes starting the FastAPI server, opening Swagger documentation, checking health, viewing scenarios, generating normal telemetry, generating replay telemetry, checking the REPLAY_DETECTED indicator, opening the Streamlit dashboard, generating events and exporting CSV data.

The final verification documentation covers the scenario set, API behaviour, replay attack behaviour, Streamlit interface, automated tests, security checks and documentation status.

30. GitHub and Version Control

Development was performed on the week6-product-development branch. The branch was pushed to GitHub and the project changes were later merged into main through a pull request. The final working tree was reported clean and the branch was synchronized with its remote before/after the merge process.

Git was used to maintain a traceable project history and to store the source code and documentation.

31. Project Limitations

The simulator does not provide real MQTT transport, a physical BMS, real sensor acquisition or production-grade cybersecurity. Cross-testing with an external developer and the SIM-010 integration harness was not actually performed and should not be represented as completed evidence.

The project is therefore best understood as a local telemetry-generation and anomaly-simulation prototype.

32. Future Improvements

Potential future improvements include real MQTT broker integration, stronger schema validation, expanded integration tests, automated replay-detection rules, richer dashboard visualizations, performance testing, more detailed telemetry persistence and broader interoperability testing.

These are future enhancements rather than claims about the current implementation.

33. Internship Learning Outcomes

The project provided practical exposure to Python modular programming, Pydantic data models, REST API development with FastAPI, Streamlit UI development, JSON telemetry structures, scenario-based testing, pytest, coverage analysis, Git/GitHub workflows and technical documentation.

The project also provided experience in presenting a cybersecurity-related concept through safe simulated data rather than real attack traffic.

34. Final Project Status

The implementation and documentation work completed for the project includes the core telemetry generator, seven scenarios, service layer, FastAPI API, Streamlit dashboard, automated tests, QA/security documentation, final verification material and release documentation. The project branch and pull request workflow were completed, with the pull request merged into main.

The demo video was recorded. Screenshots were captured for the evidence set; uploading/linking those evidence files is a separate submission-housekeeping step and is not a new development feature.

35. Conclusion

SIM-003 – MQTT Telemetry Simulator demonstrates how battery telemetry can be generated and tested in a controlled software environment. The project provides normal and abnormal telemetry scenarios, a reusable service layer, a REST API, a dashboard and automated verification.

The final recorded test result of 19 passed tests with 71% coverage demonstrates that the implemented core functionality was exercised by automated tests. The project documentation provides a structured record of architecture, design, API behaviour, QA, security, verification and release information.

The simulator can serve as a foundation for future integration with a real MQTT test environment and more advanced telemetry trust and replay-detection mechanisms.

Appendix A – Key Project Structure

models.py – BatteryData model

generator.py – telemetry generator

scenarios.py – seven scenario engine

service.py – shared service layer

api.py – FastAPI application

app.py – Streamlit dashboard

main.py – command-line generation

test/ – automated tests

FINAL_EVIDENCE/ – final evidence documents and screenshots

Appendix B – Key Test Result

pytest -q: 19 passed, 1 warning.

pytest --cov=. --cov-report=term-missing: 19 passed, 71% total coverage.

Appendix C – Key Replay Request

POST /generate-events

{"battery_id":"BAT001","scenario":"replay_attack","count":2,"delay_seconds":10}

Expected replay indicators: anomaly = REPLAY_DETECTED and simulated = true.